

TickStream

A real time streaming data platform for live crypto markets, built to run on a single laptop with one command, using only free public data.

by **Linga Reddy Gudisha**

Code: github.com/lithin45/tickstream-streaming-lakehouse

Live demo: cryptostreaming.streamlit.app

TickStream is a small but complete data platform. It listens to a live feed of cryptocurrency trades, does the kind of real time number crunching that trading desks rely on, checks every record for quality problems, and saves clean, ready to query tables that feed a live dashboard. The whole thing runs with one command, uses only free public data, and needs no accounts and no secret keys. I built it to show that I can design and ship a production shaped data system from start to finish, not just a notebook.

The problem it solves

Modern companies do not wait for data. Trading firms, ride sharing apps, banks watching for fraud, and streaming services all act on information the moment it arrives. The engineering behind that is called real time data processing, and it is one of the most in demand skills in data engineering today. TickStream is my hands on version of that whole stack, sized so it runs anywhere and anyone can watch it work in a couple of minutes.

How it works, following one trade from start to finish

- 1 **A trade happens on the exchange.** A free public feed from Coinbase broadcasts every trade and price change. My program listens and writes each one into a single tidy format.
- 2 **It lands on a fast queue.** Everything goes onto a streaming message system that keeps the flood of messages in order, so nothing is lost even when thousands arrive every minute.
- 3 **A live calculator keeps running totals.** As trades stream by, it works out, for each coin, the average price weighted by trade size, the total volume, the number of trades, and the gap between buy and sell prices. It does this in rolling one minute and five minute buckets, and it handles trades that arrive late or out of order.
- 4 **The data is filed from messy to clean.** I use a three layer pattern. The first layer is the raw data exactly as it arrived. The second is cleaned and free of duplicates. The third is the final polished summary tables. The cleaning rules are written in plain SQL.
- 5 **The clean tables remember their own history.** The final tables use a modern format that can show exactly what the data looked like at an earlier moment. People call this time travel, and it is useful for audits and debugging.
- 6 **A quality inspector guards the gate.** Before any record reaches the final tables, a data contract checks it. Price must be positive, sizes cannot be negative, the coin must be one I track. Anything that fails is set aside in a quarantine area, so it never reaches the results.
- 7 **Promises are checked automatically.** The system makes three promises and proves them on every run. Results are ready in under sixty seconds, it captures at least ninety nine percent of every minute for every coin, and zero bad records reach the final tables. On the latest run it passed all three, with results ready in about thirty seconds and full coverage.
- 8 **A live dashboard shows it all.** A web page plots the prices, volumes, and spreads for each coin, shows the quality scorecard, and lets you look back at an earlier snapshot.

One more piece I am proud of: I recorded a short slice of the real market into a file, so the entire system can be replayed offline with no internet and no keys. Anyone can reproduce my results exactly, and my automated tests run against that recording instead of a live connection, so they give the same answer every time.

The shape of the system



Quality: bad records are quarantined before they ever reach the gold layer. **Reproducible:** a recorded sample lets the whole pipeline replay offline, with no network.

The tools I used, and why

Tool	What it does here
Redpanda	The streaming queue. It speaks the industry standard Kafka language but runs as one lightweight piece, so real streaming works on a laptop.
Quix Streams	The real time calculator. A Python library that processes data as it flows, which is what computes the rolling price and volume numbers.
Apache Iceberg	The smart table format for the gold layer. It gives history, time travel, and safe changes to the table over time.
dbt with DuckDB	The SQL engine and transformation tool. It turns the raw layers into clean tables using readable SQL, with no heavy infrastructure.
Pandera	The data contract. It defines what a valid record looks like and separates the good records from the bad ones.
Streamlit and Plotly	The dashboard. They turn the gold tables into live, interactive charts.
Docker, pytest, GitHub Actions	Packaging and testing. The whole stack starts with one command and is checked automatically on every change.

What it achieves

about 30s

time from a trade arriving to it showing in the gold tables (target under 60s)

100%

of expected one minute windows captured for every coin (target 99 percent or more)

0

bad records reaching the gold tables (they are quarantined instead)

67

automated tests, all passing, run offline against the recording

What this project says about me

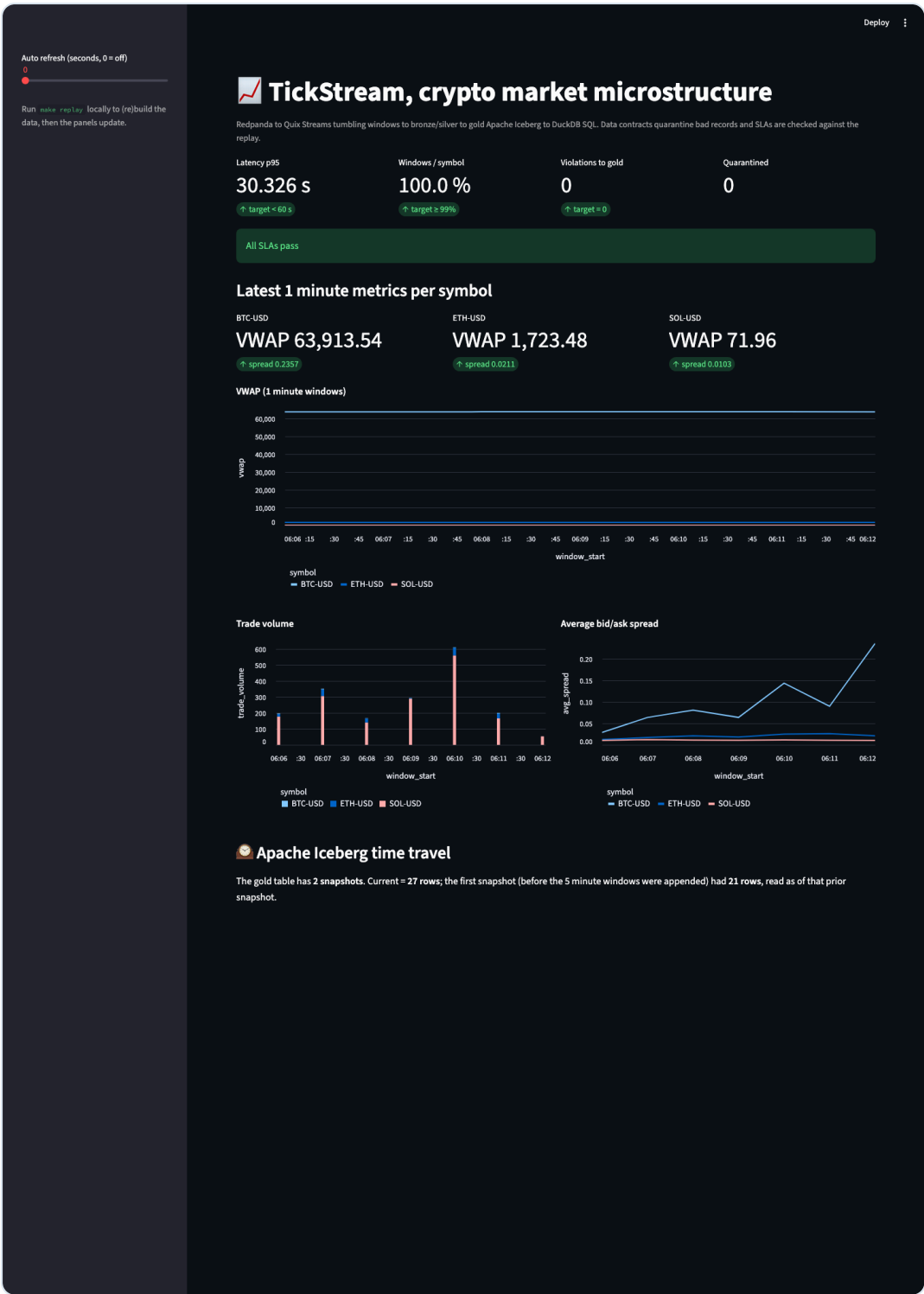
Building TickStream meant making real engineering decisions and tying many moving parts into one reliable system. It shows I can work across the whole data engineering stack: live streaming, storage, SQL modeling, data quality, testing, packaging, and visualization. More than any single tool, it shows I can take a broad goal and deliver something that actually runs, that I can explain in plain language, and that someone else can pick up and reproduce in minutes.

Run it yourself in three commands

Anyone with Docker can reproduce everything in this document, offline, in a couple of minutes. No accounts, no keys, no cloud.

<code>make up</code>	starts the streaming broker
<code>make replay</code>	builds the whole pipeline from the recorded sample and checks the quality promises
<code>make dashboard</code>	opens the live dashboard in your browser

The dashboard



The live dashboard: the quality scorecard at the top, then per coin price, volume, and spread, and a time travel panel at the bottom.

Linga Reddy Gudisha · Code and full write up: github.com/lithin45/tickstream-streaming-lakehouse · Thanks for reading.